



Módulo 5

CI/CD y Automatización

Curso: Terraform hands-on para GCP

120 min

GitOps para IaC · Terraform Cloud · GitHub Actions · OPA

Objetivos de Aprendizaje

- Explicar los modelos de ejecución de Terraform (local, TFC, self-hosted) y cuándo usar cada uno.
- Configurar un workspace en Terraform Cloud vinculado al bucket GCS del M1.
- Construir un pipeline de GitHub Actions que ejecute `fmt` + `validate` + `plan` en PRs.
- Configurar Workload Identity Federation para autenticar GitHub Actions contra GCP sin JSON keys.
- Añadir un gate de apply manual mediante GitHub Environments en merges a `main`.
- Definir y ejecutar políticas OPA/Conftest que bloqueen patrones anti-patrón en el plan.

1. Terraform Cloud: modelos de ejecución

≈ 12 min · ¿dónde corre el plan y dónde corre el apply?

Modelos de ejecución

- **Local:** CLI en la laptop del alumno. Sin concurrencia segura. Insostenible en equipo.
- **Terraform Cloud (SaaS):** runs en infraestructura de HashiCorp. Workspaces por entorno, plan/apply remoto, logs centralizados, variables cifradas.
- **Self-hosted agents** (TFE / HCP Terraform agents): runs en nuestra VPC. Útil cuando el plan necesita hablar con recursos privados.

Idea clave: la pregunta no es "qué provider uso" sino *dónde corre el run y dónde vive el state*.

Dónde corre cada cosa

Modelos de ejecución de Terraform

¿Dónde corre el plan y dónde corre el apply?

Local (CLI)

Laptop del alumno

plan
apply

state
.tfstate local

- × sin concurrencia segura
- × no audita
- ✓ barato
- ✓ válido para aprender

M1-M4 del curso

Terraform Cloud (SaaS)

Runs gestionados por HashiCorp

plan
apply

state
interno o GCS

- ✓ runs auditables
- ✓ variables cifradas
- ✓ workspaces / env
- ✓ drift detection

AppLocker usa TFC + GCS

Self-hosted agents

TFE · HCP agents en nuestra VPC

plan
apply

state
en VPC interna

- ✓ recursos privados
- ✓ VPC-SC compliant
- × costes de infra
- × op. adicional

Cuando el plan no puede salir

AppLocker: elegimos SaaS (TFC) + state delegado a GCS

Workspaces en TFC

- Cada workspace = un entorno lógico (`dev` , `pre` , `pro`).
- State remoto en TFC **o delegado al bucket GCS** del M1 (lo que usaremos en AppLocker).
- Vinculación al repo: trigger mode, branch filter, working directory.
- Ejecución: *speculative plan* en PR, *real apply* tras confirmación.

Buena práctica: el state permanece en GCP; TFC solo orquesta runs y guarda logs.

Variables y secretos

- **Terraform variables:** editables en el workspace (sensitive oculto en UI).
- **Environment variables:** para credenciales inyectadas a providers.
- Jerarquía: workspace-level > variable-set > CLI vars.
- TFC **no debe** guardar credenciales GCP de larga duración. Se prefiere **WIF** (sección 2).

Antipatrón: guardar una GCP service account JSON key en el workspace como `sensitive`. Si el SaaS se ve comprometido, esa key vive para siempre.

TFC vs. solo GitHub Actions

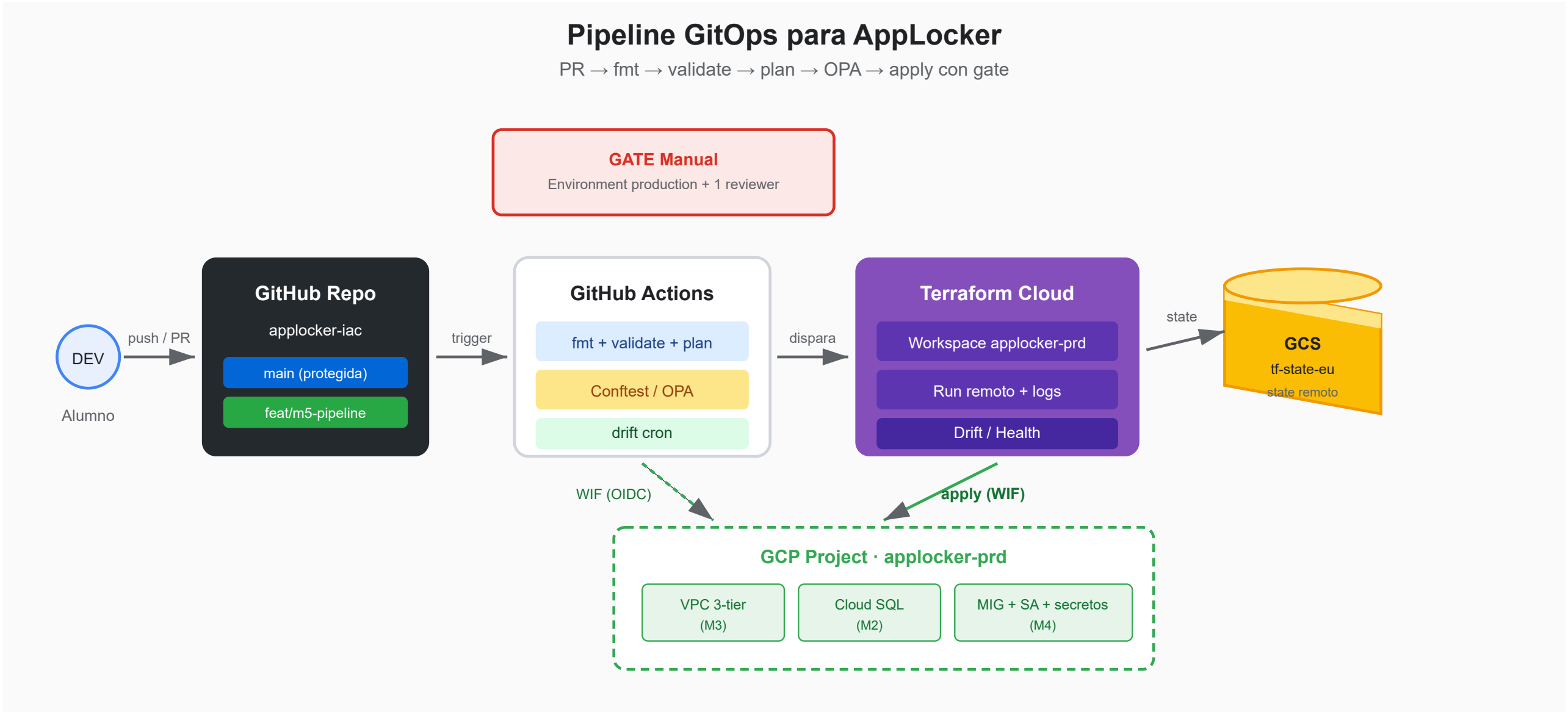
Escenario	TFC	Solo GHA
Run histórico auditable de planes y applies	✓	parcial
Drift detection programado nativo	✓ (Health)	con cron
Equipo 100% GitHub y costos ajustados	—	✓
State locking entre muchos devs	✓	parcial
Gobierno centralizado de variables y Sentinel	✓	con OPA externo

Para AppLocker: **TFC para el workspace + state + runs**, y **GitHub Actions como disparador** del workflow GitOps.

2. GitHub Actions para Terraform

≈ 13 min · motor del pipeline GitOps

Visión general del pipeline GitOps



Tres workflows, una familia

- `terraform-plan.yml` : se dispara en `pull_request` . Solo `fmt` + `validate` + `plan` + Conftest.
- `terraform-apply.yml` : se dispara en `push` a `main` o manualmente. Aplica con gate.
- `terraform-drift.yml` : `schedule` (cron) que corre `plan` y abre issue si hay drift.

Setup: `hashicorp/setup-terraform@v3` cachea providers y binarios.

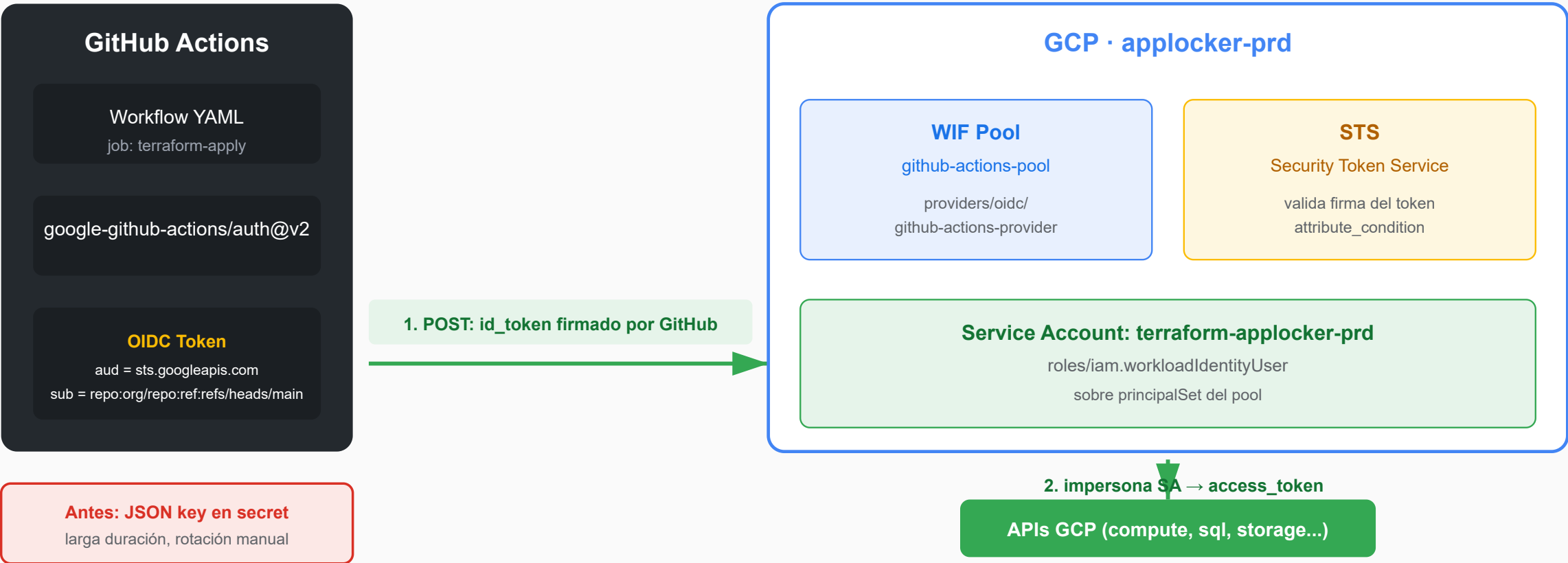
Autenticación: JSON keys vs. WIF

- **JSON keys:** larga duración, rotación manual, riesgo si se filtran. **Nunca en producción.**
- **Workload Identity Federation (WIF):** GitHub Actions asume una SA de GCP vía un pool de identidades y un provider OIDC.
- Sin secretos persistentes. Rotación automática porque cada run emite un token nuevo.

Cómo funciona WIF

Workload Identity Federation

GitHub Actions asume una SA de GCP vía OIDC — sin JSON keys



Setup de WIF (resumen)

1. Crear **Workload Identity Pool** y **Provider OIDC** en GCP apuntando a `token.actions.githubusercontent.com`.
2. Conceder `roles/iam.workloadIdentityUser` a la SA de Terraform sobre el principal GitHub (`repo:<org>/<repo>:ref:refs/heads/main`).
3. El workflow obtiene un token de GCP con `google-github-actions/auth@v2`.

Tras esto, GitHub Secrets solo guarda referencias OIDC. **Nada de JSON keys.**

Pasos del job de plan

- `terraform fmt -check -recursive` — falla si hay código mal formateado.
- `terraform init -backend=false` o contra el bucket GCS.
- `terraform validate` — sintaxis y referencias internas.
- `terraform plan -out=tfplan` + `terraform show -json tfplan > tfplan.json`.
- Publicar el plan como artifact del PR y comentar resumen automáticamente.

Gates con GitHub Environments

- **GitHub Environments:** `staging`, `production`. Definen reviewers y variables por entorno.
- El job de *apply* requiere aprobación manual del environment `production`.
- Branch protection en `main`: PR + checks verdes + 1 aprobación.

Si el plan cambia después del último check verde, GitHub invalida la aprobación y vuelve a pedirla — **idempotencia del gate**.

Drift detection programado

- Cron diario (`cron: '0 6 * * 1-5'`).
- Corre `terraform plan` y abre un issue automático si el plan no está vacío.
- Complementa TFC Health y sirve de red de seguridad si nadie mira TFC.

Por qué importa: la consola y el `kubect1 apply` de un viernes por la noche son la fuente número uno de drift. Si nadie mira, el siguiente plan te engaña.

3. Policy as Code con OPA

≈ 10 min · reglas de cumplimiento, evaluadas contra el plan

Qué es Policy as Code

- Reglas de compliance, seguridad y buenas prácticas escritas como código, evaluadas automáticamente contra el plan.
- Separa **qué se permite** (política) de **cómo se implementa** (Terraform).
- Evita depender de la revisión humana para cosas objetivas (regiones, labels, tipos de máquina).

Filosofía: las políticas son *datos*, no *código de aplicación*. Si tu política necesita un `if/else` complejo, probablemente modelas mal el problema.

Casos de uso típicos

- Bloquear regiones no permitidas (`us-central1` solo).
- Forzar labels obligatorios (`env` , `owner` , `project`).
- Impedir tipos de máquina caros o no soportados (`n1-standard-96` fuera).
- Obligar `deletion_protection = true` en Cloud SQL.
- Impedir buckets públicos o IPs externas en recursos internos.

OPA evalúa el plan

Policy as Code con OPA

Rego evalúa el plan JSON — deny[msg] bloquea antes del apply



`conftest test tfplan.json --policy policies/` — corre local y en CI con el mismo binario

Rego: el lenguaje de OPA

- Sintaxis declarativa. Evalúa un input (el plan JSON) contra reglas.

Estructura básica:

- `package terraform.policies`
- `deny[msg] { ... condición; msg := sprintf("...", [...]) }`

- Un `deny` con cualquier mensaje → falla la política.

Tip: en play.openpolicyagent.org puedes pegar un plan JSON de ejemplo y validar tus reglas sin tocar el repo.

Integración con pipelines: Conftest

- **Conftest:** CLI que evalúa políticas Rego contra archivos JSON (plan de Terraform).
- Flujo: `terraform show -json plan > plan.json` → `conftest test plan.json`.
- Corre **antes** del apply. Si falla, el apply no se ejecuta.
- Alternativas: Sentinel (propietario, solo en TFC), Checkov, Terrascan.

4. Lab: Pipeline GitOps para AppLocker

≈ 75 min · guiado por el formador

Prerrequisitos del lab

- Repositorio Git `applocker-iac` con todo el código de M1–M4.
- Bucket GCS `tf-state-applocker-eu-<sufijo>` (M1) accesible.
- SA `terraform-applocker-prd` con permisos sobre el proyecto (M4).
- Secretos en Secret Manager (M4): `db-master-password`, `app-api-key`.
- Cuenta TFC con organización `applocker-<sufijo>`.

4.2 — Workspace en TFC

1. Crear workspace `applocker-prd` con execution mode `Remote` .
2. Conectar al repo GitHub `applocker-iac` :
 3. Trigger: `Push to branches` = `main` , PR activado.
 4. Working directory: `./envs/prd` .
5. En **Variables**, definir: `TF_VAR_project_id` , `TF_VAR_region` .

```
terraform {  
  backend "gcs" {  
    bucket = "tf-state-applocker-eu-<sufijo>"  
    prefix = "envs/prd"  
  }  
}
```

4.3 — Workload Identity Federation

```
PROJECT_ID="applocker-prd"
POOL_ID="github-actions-pool"
PROVIDER_ID="github-actions-provider"
SA="terraform-applocker-prd@${PROJECT_ID}.iam.gserviceaccount.com"
GITHUB_REPO="mi-org/applocker-iac"

# 1. Crear el pool
gcloud iam workload-identity-pools create "${POOL_ID}" \
  --project="${PROJECT_ID}" --location="global" \
  --display-name="GitHub Actions Pool"

# 2. Crear el provider OIDC
gcloud iam workload-identity-pools providers create-oidc "${PROVIDER_ID}" \
  --project="${PROJECT_ID}" --location="global" \
  --workload-identity-pool="${POOL_ID}" \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub,attribute.actor=assertion.actor,attribute.repository=assertion.repository" \
  --attribute-condition="assertion.repository=='${GITHUB_REPO}'"
```

GitHub Secrets para el repo

Secret	Valor
GCP_WORKLOAD_IDENTITY_PROVIDER	projects/<PROJECT_NUMBER>/locations/global/workloadIdentityPools/github-actions-pool/providers/github-actions-provider
GCP_SERVICE_ACCOUNT	terraform-applocker-prd@applocker-prd.iam.gserviceaccount.com
TF_API_TOKEN	Token de TFC (user token)
TF_ORGANIZATION	applocker-<sufijo>

Ya no hay JSON keys en GitHub Secrets. Solo referencias OIDC.

4.4 — Workflow terraform-plan.yml (PR)

```
name: "Terraform Plan"

on:
  pull_request:
    branches: [main]
    paths:
      - 'envs/**'
      - 'modules/**'
      - '.github/workflows/**'

permissions:
  id-token: write # obligatorio para WIF
  contents: read
  pull-requests: write

jobs:
  terraform-plan:
    runs-on: ubuntu-latest
    environment: staging
```

4.5 — Workflow terraform-apply.yml (gate)

```
name: "Terraform Apply"

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

jobs:
  terraform-apply:
    runs-on: ubuntu-latest
    environment: production # gate: 1 aprobador obligatorio
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
```

GitHub: `Settings > Environments > production > Required reviewers = 1` . Sin ese aprobador, el apply **nunca** se ejecuta.

4.6 — Workflow drift detection

```
name: "Terraform Drift Detection"

on:
  schedule:
    - cron: '0 6 * * 1-5'    # L-V a las 06:00 UTC
  workflow_dispatch:

permissions:
  id-token: write
  contents: read
  issues: write

jobs:
  drift-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
```

4.7 — Política de regiones

```
# envs/prd/policies/region.rego
package terraform.policies.region

deny[msg] {
  rc := input.resource_changes[_]
  rc.change.actions[_] != "delete"
  rc.change.after.region != "us-central1"
  rc.change.after.region != "europe-west3"
  rc.type != "google_client_config"
  msg := sprintf("%s está en región no permitida: %v", [rc.address, rc.change.after.region])
}
```


4.7 — Labels obligatorios

```
# envs/prd/policies/required_labels.rego
package terraform.policies.labels

required_labels := {"env", "owner", "project", "managed_by"}

deny[msg] {
    rc := input.resource_changes[_]
    rc.change.actions[_] != "delete"
    labels := object.get(rc.change.after, "labels", {})
    missing := required_labels - object.keys(labels)
    count(missing) > 0
    msg := sprintf("%s le faltan labels obligatorios: %v", [rc.address, missing])
}
```

4.7 — Cloud SQL protegida

```
# envs/prd/policies/sql_protection.rego
package terraform.policies.cloudsql

deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "google_sql_database_instance"
  rc.change.actions[_] != "delete"
  rc.change.after.deletion_protection != true
  msg := sprintf("%s debe tener deletion_protection = true", [rc.address])
}

deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "google_sql_database_instance"
  rc.change.actions[_] != "delete"
  rc.change.after.ip_configuration[_].ipv4_enabled == true
  msg := sprintf("%s no debe tener IP pública habilitada", [rc.address])
}
```

4.7 — Storage y tipos de máquina

```
# envs/prd/policies/storage.rego
package terraform.policies.storage

deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "google_storage_bucket"
  rc.change.actions[_] != "delete"
  rc.change.after.iam[_].member == "allUsers"
  msg := sprintf("%s expone el bucket a allUsers", [rc.address])
}

# envs/prd/policies/machine_types.rego
package terraform.policies.machine_types

banned := {"n1-highmem-96", "n1-standard-96", "m1-ultramem-40"}

deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "google_compute_instance_template"
```

Cinco políticas cubren los anti-patrones más comunes. Cualquiera nueva se mete como un archivo `.rego` más en `policies/`.

4.8 — Verificación local de Conftest

```
cd envs/prd
terraform plan -out=tfplan
terraform show -json tfplan > tfplan.json
conftest test tfplan.json --policy policies/
```

Resultado esperado: si todo cumple, `0 failures`. Si falla, Conftest lista los `deny`.

El mismo comando corre dentro de los workflows. **Plan local = plan CI.**

4.9 — Validación final (con la clase)

1. Crear `feat/m5-pipeline` y modificar un label de un recurso (debe pasar OPA).
2. Borrar un label obligatorio (debe fallar OPA).
3. Abrir PR → comprobar que corre `terraform-plan.yml`, publica artifact y comenta en el PR.
4. Aprobar el PR y mergear a `main` → comprobar `terraform-apply.yml`, el gate de `production` y que el apply va contra el bucket GCS.
5. Disparar `terraform-drift.yml` manualmente y abrir el issue.

4.10 — Limpieza del lab

```
# NO destruir infra – solo limpiar artefactos del lab
gh workflow disable terraform-drift.yml || true
rm -rf .github/workflows/terraform-drift.yml
git checkout main
git pull
```

Recordatorio: los workflows `terraform-plan.yml` y `terraform-apply.yml` quedan activos. El de drift se elimina porque lo reemplaza TFC Health en M6.

5. Cierre, Q&A y M6

≈ 5 min

Recap del módulo

- Tres pilares montados: **Terraform Cloud** (orquestración y state remoto en bucket GCS), **GitHub Actions** (motor del pipeline), **OPA + Conftest** (gate de calidad).
- **WIF** reemplaza las JSON keys. Ya no hay secretos persistentes en GitHub.
- Cualquier cambio a AppLocker pasa por PR + plan visible + OPA verde + apply con gate humano.

Adelanto de M6 — Observabilidad y Operación

- **Monitoring:** métricas con Cloud Monitoring y dashboards para Cloud SQL, MIG y balanceador.
- **Logging:** centralización en Cloud Logging con sinks y filtros.
- **Alerting:** políticas de alerta basadas en SLI/SLO.
- **Cost control:** budgets y alertas de facturación por entorno.
- **Runbooks:** respuesta a incidentes usando los logs y métricas que ahora existen.

La infra endurecida del M4 y los gates del M5 hacen que M6 se enfoque en **operar**, no en **reparar**.

Resumen

Puntos Clave

- GitOps para IaC = **TFC + GitHub Actions + OPA**. Los tres son necesarios, ninguno solo cubre el ciclo.
- **TFC** orquesta runs y delega state a GCS; los workspaces representan entornos.
- **WIF** elimina JSON keys: GitHub Actions asume la SA de Terraform vía OIDC.
- **GitHub Environments** = gate de apply manual por entorno, integrado con branch protection.
- **OPA + Conftest** ejecutan políticas Rego contra el plan JSON antes del apply.
- **Drift detection** programado (cron o TFC Health) detecta cambios hechos por fuera de Terraform.

Referencias

Recursos Adicionales

- [Terraform Cloud — Overview](#)
- [Terraform Cloud — Workspaces](#)
- [GitHub Actions para Terraform](#)
- [GCP Workload Identity Federation](#)
- [GitHub Actions — OIDC con GCP](#)
- [GitHub Environments](#)
- [Open Policy Agent — Documentación](#)
- [Conftest](#)
- [Rego Playground](#)
- [Terraform fmt / validate / plan](#)



Gracias!

Módulo 5 Completado

Siguiente: M6 — Observabilidad y Operación